



Software Requirements Specification

Doctor–Patient Dispensary Queue Management Platform

Sri Lanka | Mobile App + Web Portal

Document Title	OneHealth – Software Requirements Specification (SRS)
Product Name	OneHealth
Version	1.0
Date	12 July 2026
Prepared For	Real-world production deployment (rapid MVP)
Target Delivery Team	2 Full-Stack Developers
Classification	Confidential – Internal & Investor Use

Document Control

Revision History

Version	Date	Author	Description
0.1	05 Jul 2026	Product Owner	Initial draft based on field problem statement gathered in Sri Lanka.
1.0	12 Jul 2026	OneHealth Project Team	First complete SRS: use case/ER/EER/class/activity models, functional & non-functional requirements, architecture, and 2-developer delivery plan.

Distribution

- Founding team / product owner
- Development team (Developer A – Backend, Developer B – Frontend)
- Prospective doctors / pilot dispensaries during onboarding

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements, system models, architecture, and delivery plan for OneHealth — a mobile and web platform that connects patients with private-practice doctors (consultants) and their dispensaries in Sri Lanka. The document is written to be directly usable by a two-person development team to design, build, and deploy a production system; it is not an academic or university-project specification.

1.2 Scope

OneHealth allows a doctor to register a private dispensary (channelling centre) on the platform after identity and credential verification, publish its location and operating hours, and let patients discover the dispensary, view ratings/reviews, and reserve a queue (channelling) number remotely. Patients can track their queue number in real time instead of waiting physically at the dispensary or guessing when to return. The platform is offered through three subscription packages for doctors — Free, Standard, and Premium — each unlocking a progressively richer set of features (staff accounts, online payment and invoicing, e-prescriptions, and full electronic medical records). The system also includes an administrative back office for verifying doctors, moderating reviews, and managing billing, and a lightweight AI recommendation feature that suggests suitable doctors based on patient-entered symptoms and accumulated review data.

1.3 Problem Statement

The requirement originates from a real, recurring problem in Sri Lanka's private outpatient (channelling) system: a patient who is unwell — for example, with a fever — but away from their usual area does not know which nearby private medical centres are reputable. Even after locating a dispensary, the day's queue numbers may already be exhausted by the time the patient arrives, so no consultation or medicine can be obtained that day. Separately, once a number is issued, some patients leave and return later expecting to keep their place, and by the time they come back their number may already have been called and passed. OneHealth directly targets this problem by letting patients discover verified nearby doctors, see live queue status, and reserve a number before travelling, while giving doctors and their staff a simple tool to manage the queue, payments, and patient records digitally instead of relying on paper token boards or informal channels such as WhatsApp.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
Dispensary	A doctor's private consultation/channelling centre where patients are seen and, in most cases, medicine is dispensed on site.
Queue Number / Token	The sequential number assigned to a patient for a specific doctor on a specific day, used to determine consultation order.
Staff Member	A receptionist or pharmacist granted delegated access by a doctor under the Standard or Premium package.
SLMC	Sri Lanka Medical Council — the statutory body whose registration number is used to verify a doctor's licence.
e-Prescription	A digitally issued prescription sent from doctor to pharmacist/patient inside the platform instead of handwritten paper or WhatsApp.
EMR	Electronic Medical Record — a patient's diagnosis history and uploaded reports, available under the Premium package.

MVP	Minimum Viable Product — the first production-ready release scoped for rapid delivery by two developers.
SRS	Software Requirements Specification — this document.

1.5 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (structural reference only).
- Problem statement and package definitions supplied by the OneHealth product owner (July 2026).
- Sri Lanka Medical Council (SLMC) public registry — used conceptually for the doctor-verification workflow.

1.6 Document Overview

Section 2 describes the product at a high level and its user classes. Section 3 defines the three subscription packages and their feature matrix. Section 4 presents the system models — use case, activity, class, ER and EER diagrams. Section 5 lists detailed functional requirements grouped by actor. Section 6 lists non-functional requirements. Section 7 defines the system architecture, technology stack and database design. Section 8 gives a delivery plan for a two-developer team, and the appendices contain the glossary and sign-off.

2. Overall Description

2.1 Product Perspective

OneHealth is a new, standalone platform consisting of a cross-platform mobile app (patients and doctors), a responsive web portal (doctors and administrators), a backend API, and a small AI micro-service for doctor recommendation. It is not a module of an existing hospital or government system; it is built to onboard private-practice doctors independently and can later integrate with national e-health initiatives if required.

2.2 Product Features Summary

- Doctor onboarding with certificate/licence verification by an admin before the profile goes live.
- Dispensary registration with map location, city/town, and working hours.
- Patient search and discovery by doctor name, city, or town, with ratings and reviews.
- Remote queue (channelling) number booking with a live, real-time queue counter.
- Push/SMS notification as a patient's number approaches.
- Tiered packages (Free / Standard / Premium) unlocking staff accounts, online payment & invoicing, e-prescriptions, and full electronic medical records.
- Post-visit feedback that also trains an AI recommender to suggest suitable doctors from symptoms.
- Admin console for verification, review moderation, and subscription/billing management.

2.3 User Classes and Characteristics

Actor	Description	Key Characteristics
Patient	General public seeking a doctor and wanting to avoid physically waiting in a queue.	Mobile-first, low technical expertise assumed; needs a simple, fast booking flow.
Doctor	Private-practice consultant who owns one or more dispensaries and subscribes to a package.	Uses both mobile app and web portal; requires credential verification before going live.
Staff Member	Receptionist or pharmacist given delegated, permission-scoped access by a doctor (Standard/Premium only).	Operates mostly from the dispensary counter; needs a fast queue-update and payment/dispense UI.
System Admin	OneHealth back-office staff who verify doctors, moderate content, and manage billing.	Power users on the web portal; require audit trails for approval/rejection actions.

2.4 Operating Environment

- Patient/Doctor mobile app: Android 8+ and iOS 14+ via a single React Native codebase.
- Doctor/Admin web portal: modern evergreen browsers (Chrome, Edge, Safari, Firefox), responsive down to tablet width.
- Backend: containerised Node.js service deployable to any cloud VM or PaaS (Render, Railway, AWS).
- Network: designed for Sri Lankan mobile-data conditions — payloads kept small, images compressed, and offline-tolerant queue-status caching on the client.

2.5 Design and Implementation Constraints

- Must be deliverable as a rapid MVP by two developers — favour managed services and proven libraries over custom infrastructure.
- Doctor identity/credential verification is mandatory before any dispensary becomes publicly bookable (patient safety and trust requirement).



- Medical record and report data (Premium) must be stored with access control scoped to the owning patient and the treating doctor only.
- Payment processing (Standard/Premium) must go through a licensed local/international payment gateway (e.g., PayHere for LKR, or Stripe) rather than custom card handling.

2.6 Assumptions and Dependencies

- Doctors are willing to scan/upload their SLMC registration and business/medical-centre licence during onboarding.
- Patients and dispensary counters have reasonably reliable mobile internet access.
- A third-party SMS/push provider and a payment gateway account can be procured before the Standard-package launch.

3. Subscription Packages

Doctors choose one of three packages when registering a dispensary on OneHealth. Every package includes the core discovery, booking, and live-queue experience for patients; higher packages add staff delegation, payments, and clinical record-keeping.

3.1 Free Plan

Designed for a solo doctor testing the platform. The doctor is the only account holder — no staff/administrator accounts can be added. The doctor personally updates the current queue number from the mobile app or web portal. Patients can search, view the doctor's profile, ratings and reviews, and book a queue number; only in-person (cash/card at counter) payment is supported — no online payment integration at this tier.

3.2 Standard Plan (Paid)

Everything in Free, plus the ability to add staff accounts (receptionist and/or pharmacist) with scoped permissions. Staff can be delegated the right to update the live queue number on the doctor's behalf. This tier activates online payment for queue bookings, automatic e-invoice generation, and the ability for the doctor to send an e-prescription directly to the pharmacist role (removing the informal WhatsApp workaround many dispensaries currently use).

Additional Standard-tier features recommended for the real-world rollout:

- Daily/weekly booking and revenue summary for the doctor.
- Configurable daily queue capacity (maximum tokens per day) per dispensary.
- Multi-staff shift assignment (e.g., morning/evening receptionist).

3.3 Premium Plan (Paid)

Everything in Standard, plus full electronic medical record (EMR) management: the doctor can enter and view a patient's diagnosis history, and both doctor and patient can securely upload and exchange reports and documents inside the app instead of informal channels. The doctor can review a patient's past visit history at the point of consultation.

Additional Premium-tier features recommended for the real-world rollout:

- Multi-dispensary / multi-branch management for doctors who practise at more than one centre.
- Priority placement in AI doctor-recommendation results (clearly labelled as sponsored/priority, not hidden).
- Exportable patient visit summary (PDF) for referrals to other specialists.

3.4 Feature Comparison Matrix

Feature	Free	Standard (Paid)	Premium (Paid)
Doctor profile, search & discovery	✓	✓	✓
Dispensary & location management	✓	✓	✓
Patient queue booking & live number	✓	✓	✓
Ratings, reviews & AI recommendation input	✓	✓	✓
Staff / administrator accounts	—	✓	✓
Online payment & e-invoice	—	✓	✓
e-Prescription (doctor → pharmacist/patient)	—	✓	✓
Booking & revenue analytics	—	✓	✓

Patient electronic medical records (EMR)	—	—	✓
Report/document upload & secure sharing	—	—	✓
Multi-dispensary management	—	—	✓

4. System Models

This section presents the analysis and design models used to derive the requirements in Section 5: use case diagrams for each actor group, an activity diagram for the core patient booking flow, a class diagram of the domain model, and the entity-relationship (ER) and enhanced entity-relationship (EER) diagrams for the database.

4.1 Use Case Diagrams

The use case model is split into two diagrams for readability: Patient & Doctor interactions, and Staff & Admin interactions.

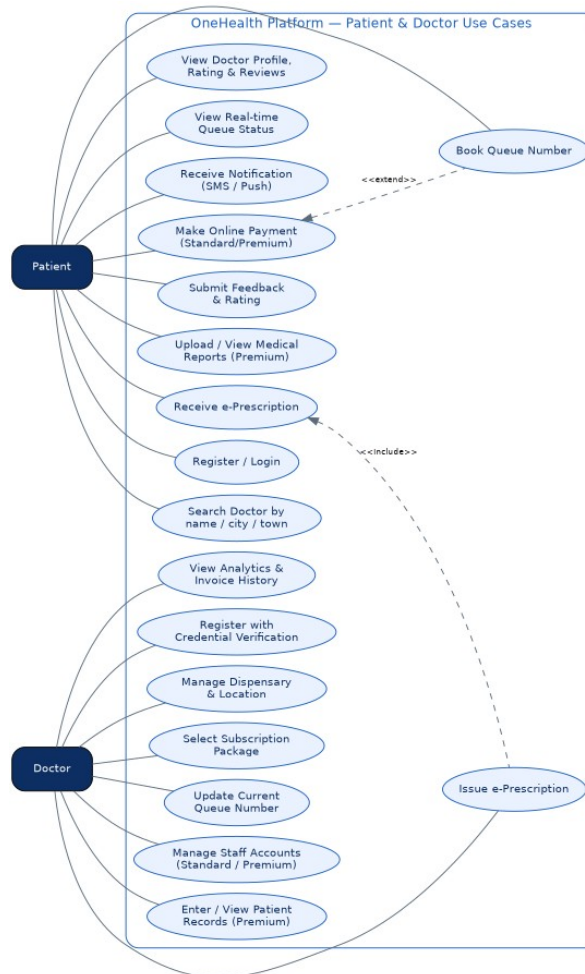


Figure 4.1 — Use Case Diagram: Patient and Doctor

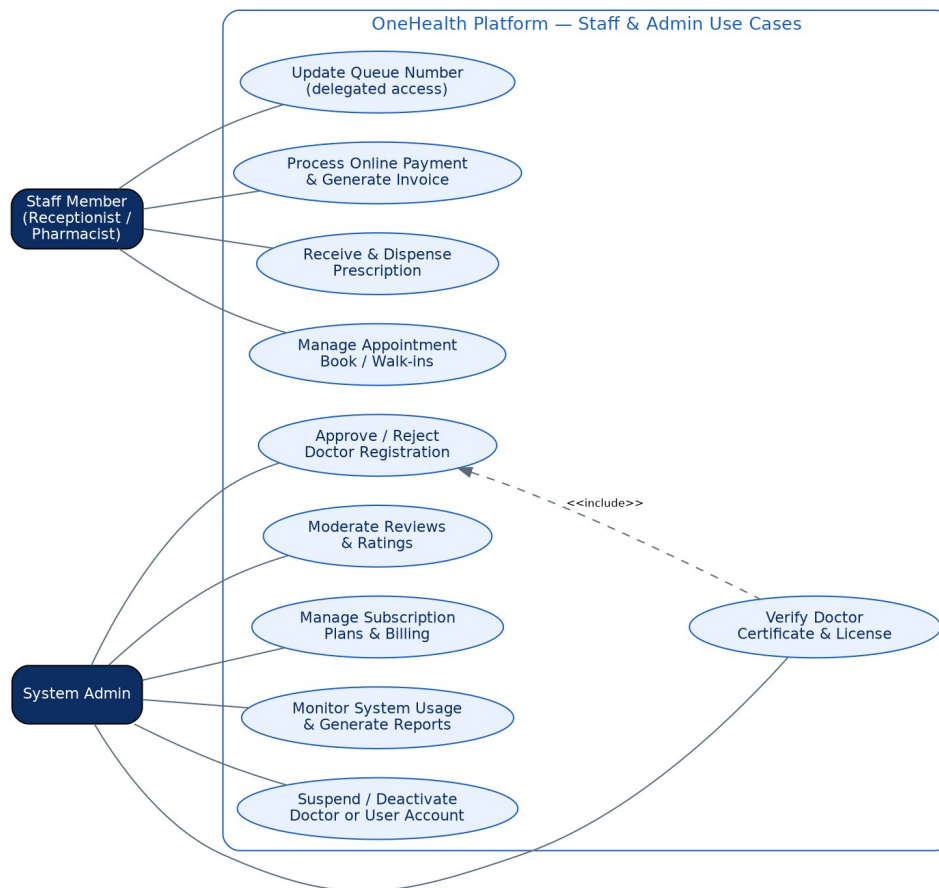


Figure 4.2 — Use Case Diagram: Staff Member and System Admin

4.1.1 Key Use Case Descriptions

Use Case	Primary Actor	Brief Description
Book Queue Number	Patient	Patient selects a doctor/dispensary and date, and reserves the next available queue token before travelling.
Update Current Queue Number	Doctor / Staff	The dispensary marks the currently-being-served token number, instantly pushed to all patients waiting for that day.
Verify Doctor Certificate & Licence	System Admin	Admin reviews the uploaded SLMC registration and medical-centre licence before approving the doctor's profile to go live.
Issue e-Prescription	Doctor	Doctor records prescribed medicines against a consultation and routes them to the pharmacist role and/or the patient.
Submit Feedback & Rating	Patient	After a completed visit, the patient rates the doctor and leaves a comment; this also becomes training data for the AI recommender.

4.2 Activity Diagram — Patient Queue Booking Flow

The activity diagram below models the exact real-world problem described in Section 1.3: a patient searching for a doctor, checking availability before travelling, booking a number, and being notified as their turn approaches — removing the need to guess or wait physically at the dispensary.

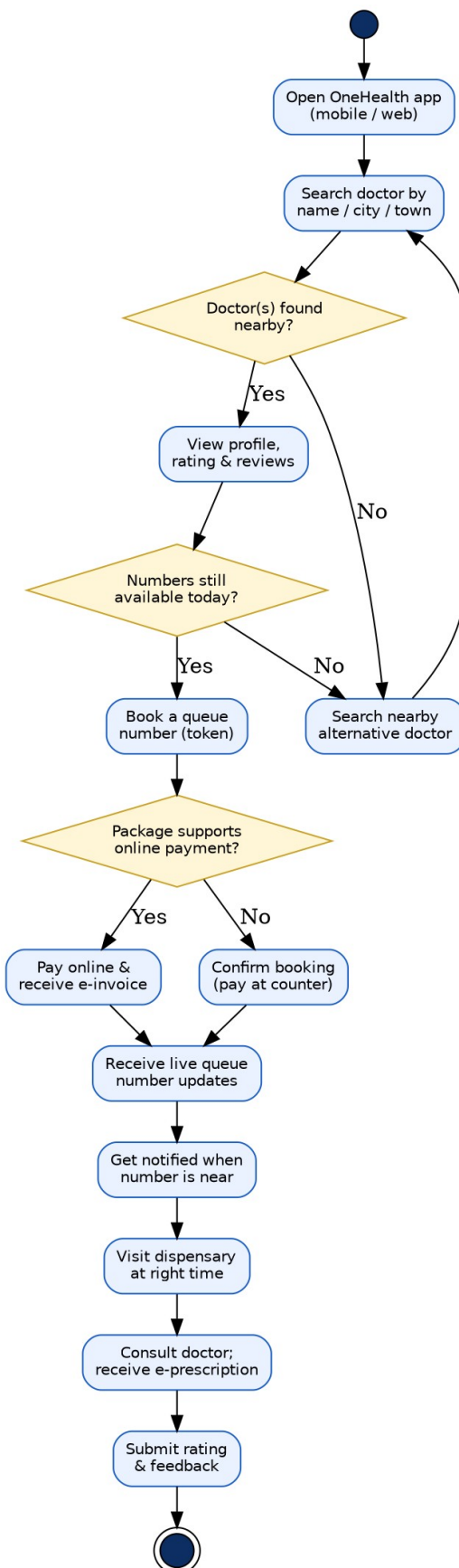


Figure 4.3 — Activity Diagram: Patient Queue Booking Flow

The domain model below centres on a base User that specialises into Doctor, Patient, Staff Member and Admin. A Doctor owns one or more Dispensaries and subscribes to a Package that governs feature access; Patients book QueueTokens against a Dispensary, which may generate a Payment/Invoice, a Prescription, and later a Review that feeds the AI Recommender Service.

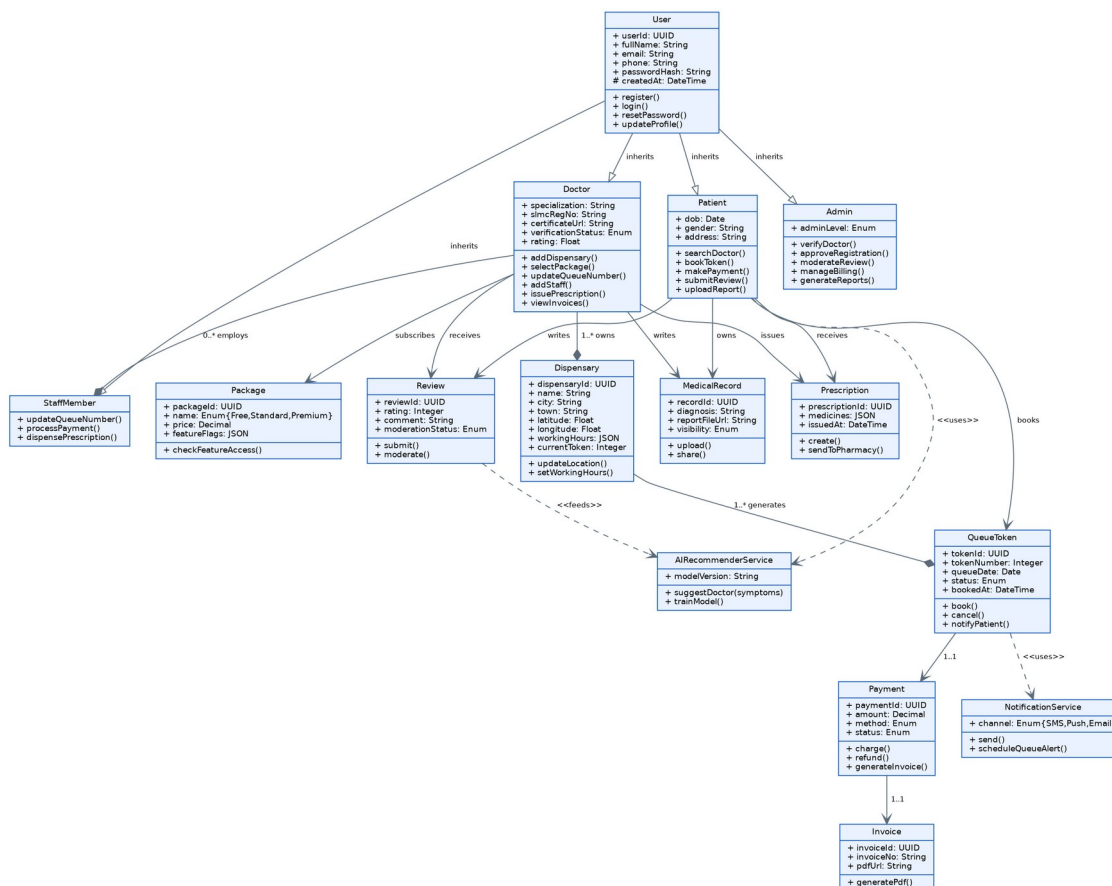


Figure 4.4 — Class Diagram: OneHealth Domain Model

4.4 Entity-Relationship (ER) Diagram

The ER diagram shows the physical relational structure that underpins the class model, including cardinalities used to derive foreign keys and indexes (see Section 7.3 for the database design).

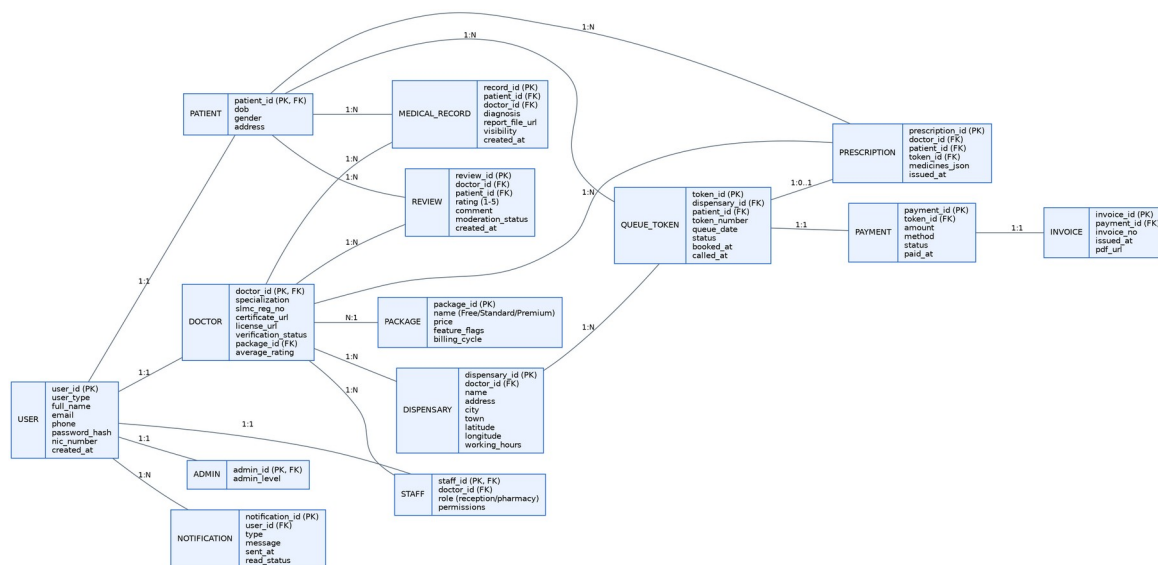


Figure 4.5 — Entity-Relationship (ER) Diagram

4.5 Enhanced Entity-Relationship (EER) Diagram

The EER diagram formalises the generalisation/specialisation of USER into DOCTOR, PATIENT, STAFF and ADMIN as a disjoint, total specialisation — every registered user is exactly one of the four subtypes, sharing common attributes (name, email, phone, password hash) in the USER superclass while each subtype carries its own attributes (e.g., a Doctor's SLMC registration number, a Patient's date of birth).

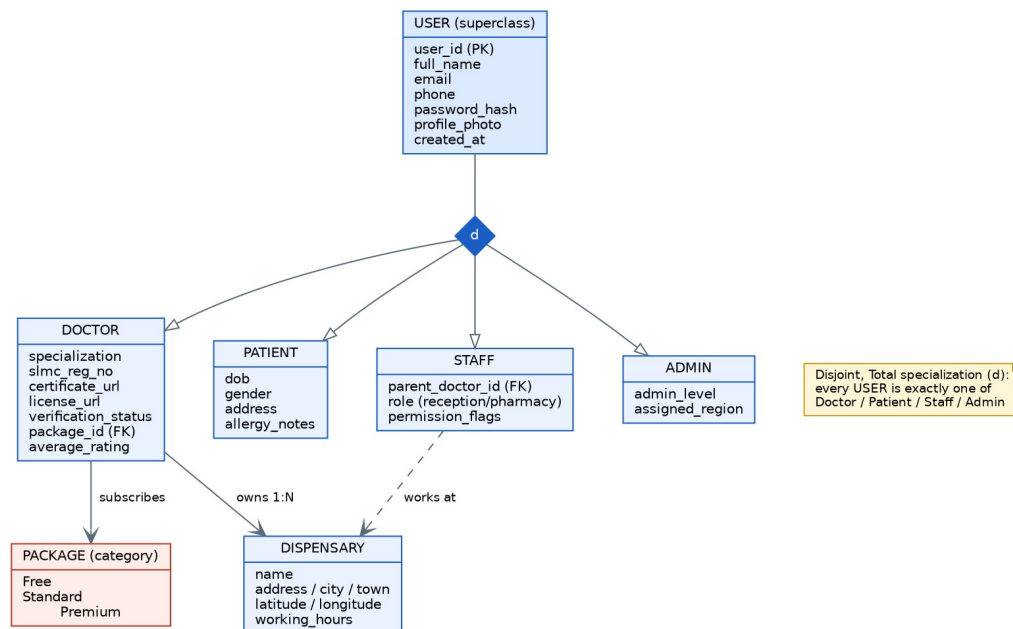


Figure 4.6 — Enhanced Entity-Relationship (EER) Diagram

5. Functional Requirements

Requirements are grouped by actor. "Package" indicates the minimum subscription tier a doctor-side requirement applies to; patient-facing requirements generally apply regardless of the doctor's package unless noted. Priority follows MoSCoW: Must, Should, Could.

5.1 Patient Functional Requirements

ID	Requirement	Package	Priority
FR-P01	The system shall allow a patient to register and log in using phone number/email and password, with OTP-based phone verification.	All	Must
FR-P02	The system shall allow a patient to search for doctors by name, city, town, or specialization.	All	Must
FR-P03	The system shall display a doctor's profile, current package-level services, average rating, and written reviews.	All	Must
FR-P04	The system shall allow a patient to book the next available queue number for a chosen doctor/dispensary and date.	All	Must
FR-P05	The system shall show the patient the live current queue number for their booked dispensary/date, refreshed in real time.	All	Must
FR-P06	The system shall send a push/SMS notification to the patient when their queue number is within a configurable threshold (e.g., 5 numbers away).	All	Should
FR-P07	The system shall allow a patient to cancel a booked queue number before it is called.	All	Should
FR-P08	The system shall allow a patient to pay online for a booking and receive a digital receipt/invoice.	Standard+	Must
FR-P09	The system shall allow a patient to receive an e-prescription issued by the doctor after consultation.	Standard+	Must
FR-P10	The system shall allow a patient to upload personal medical reports/documents and view doctor-shared reports.	Premium	Must
FR-P11	The system shall allow a patient to view their own visit and prescription history.	Premium	Should
FR-P12	The system shall allow a patient to submit a star rating and written feedback after a completed visit.	All	Must
FR-P13	The system shall let a patient describe symptoms in free text and receive AI-suggested suitable doctors nearby.	All	Could

5.2 Doctor Functional Requirements

ID	Requirement	Package	Priority
FR-D01	The system shall allow a doctor to register an account and submit SLMC registration number, professional certificate, and medical-centre licence for verification.	All	Must
FR-D02	The system shall prevent a doctor's dispensary from appearing in patient search until an admin approves the submitted credentials.	All	Must
FR-D03	The system shall allow a verified doctor to create and manage one or more dispensaries with name, address, city/town, map location, and working hours.	All / Multi-branch: Premium	Must
FR-D04	The system shall allow a doctor to select and change their	All	Must

	subscription package (Free/Standard/Premium).		
FR-D05	The system shall allow a doctor to update the current queue number from the mobile app or web portal.	All	Must
FR-D06	The system shall allow a doctor to set a maximum daily queue capacity per dispensary.	Standard+	Should
FR-D07	The system shall allow a doctor to add staff (receptionist/pharmacist) accounts and assign scoped permissions.	Standard+	Must
FR-D08	The system shall allow a doctor to issue an e-prescription against a specific patient visit and route it to a pharmacist/patient.	Standard+	Must
FR-D09	The system shall allow a doctor to view booking, revenue, and rating analytics for their dispensary.	Standard+	Should
FR-D10	The system shall allow a doctor to enter/update a patient's diagnosis record and view the patient's past visit history.	Premium	Must
FR-D11	The system shall allow a doctor to upload and securely share a report/document with a specific patient.	Premium	Must
FR-D12	The system shall allow a doctor to respond publicly to a patient review.	All	Could

5.3 Staff Member Functional Requirements

ID	Requirement	Package	Priority
FR-S01	The system shall allow a staff member to log in with credentials issued by their doctor and access only their assigned dispensary.	Standard+	Must
FR-S02	The system shall allow a staff member with reception permission to update the current queue number.	Standard+	Must
FR-S03	The system shall allow a staff member with reception permission to process an online/in-person payment and generate an invoice.	Standard+	Must
FR-S04	The system shall allow a staff member with pharmacist permission to view and mark an e-prescription as dispensed.	Standard+	Must
FR-S05	The system shall allow a staff member to view and manage walk-in patients alongside app-booked patients in a single queue view.	Standard+	Should

5.4 System Admin Functional Requirements

ID	Requirement	Package	Priority
FR-A01	The system shall allow an admin to review a doctor's submitted certificate/licence and approve or reject the registration with a reason.	All	Must
FR-A02	The system shall allow an admin to suspend or deactivate a doctor, staff, or patient account that violates platform policy.	All	Must
FR-A03	The system shall allow an admin to moderate (hide/remove) an inappropriate or fraudulent review.	All	Must
FR-A04	The system shall allow an admin to manage subscription package pricing and view billing history across doctors.	All	Should
FR-A05	The system shall allow an admin to view platform-wide usage reports (active doctors, bookings, revenue).	All	Should
FR-A06	The system shall allow an admin to view an audit log of approval/rejection/suspension actions.	All	Should

5.5 AI Recommendation Feature Requirements

ID	Requirement	Package	Priority
FR-AI01	The system shall collect structured feedback (rating, comment, specialization, symptoms if provided) after each completed visit as training data.	All	Must
FR-AI02	The system shall provide a symptom-input screen that returns a ranked list of nearby, verified doctors suited to the described symptoms.	All	Could
FR-AI03	The AI recommendation shall never replace or claim to give medical diagnosis — it shall only suggest which doctor/specialization to consult.	All	Must

6. Non-Functional Requirements

Category	Requirement
Performance	Queue-status updates must propagate to a patient's device within 3 seconds of a staff/doctor update, via the real-time (Socket.IO) channel. Search results shall return within 2 seconds under normal 3G/4G conditions.
Scalability	The backend shall be stateless and horizontally scalable behind the API gateway; Redis shall back the real-time layer so multiple backend instances can share live queue state.
Availability	Target 99.5% uptime for the booking and live-queue services during business hours (7am–9pm local time), the busiest period for private dispensaries.
Security	Passwords stored with bcrypt/argon2 hashing; all traffic over HTTPS/TLS; JWT access tokens with short expiry plus refresh tokens; role-based access control enforced on every API route.
Data Privacy	Medical records and reports (Premium) are accessible only to the owning patient and their treating doctor; all access is logged. This aligns with the sensitivity expected for personal health information even ahead of any formal certification.
Usability	The patient booking flow (search → view profile → book number) must be completable in under 5 taps/screens on the mobile app, since the target user may be unwell and have low patience for a complex flow.
Localization	UI text externalised for English, Sinhala, and Tamil from the first release, since dispensaries and patients across Sri Lanka use all three languages.
Reliability	Queue-number updates and payment transactions must be idempotent and resistant to double-submission (e.g., a patient tapping "Book" twice must not create two tokens).
Maintainability	Codebase organised into clear modules per domain (auth, queue, payments, records) as detailed in the accompanying File & Development Structure annex, enabling two developers to work in parallel with minimal merge conflicts.
Compatibility	Mobile app supports Android 8+ and iOS 14+; web portal supports the last two major versions of Chrome, Edge, Safari, and Firefox.
Auditability	All admin actions (approve/reject doctor, suspend account, moderate review) are recorded with timestamp and admin identity for dispute resolution.

7. System Architecture

The architecture favours managed services and well-documented, mainstream libraries so that a two-developer team can ship a production MVP quickly without operating heavy custom infrastructure.

7.1 Architecture Overview

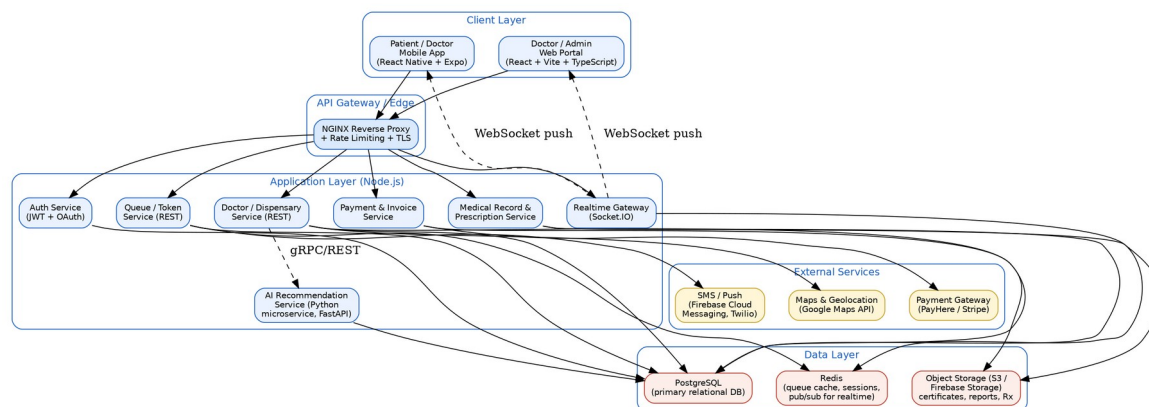


Figure 7.1 — OneHealth System Architecture

Both the mobile app and web portal talk to a single NGINX gateway in front of a Node.js application layer split into domain services (auth, queue, doctor/dispensary, payments, records). A Socket.IO realtime gateway, backed by Redis pub/sub, pushes live queue-number updates to connected devices. PostgreSQL is the system of record; an object store (S3-compatible or Firebase Storage) holds certificates, uploaded reports, and prescriptions. A small Python/FastAPI micro-service handles the AI doctor-recommendation feature independently of the core booking flow so it can be iterated on without touching booking-critical code.

7.2 Technology Stack

Layer	Technology	Rationale
Patient/Doctor mobile app	React Native + Expo, TypeScript	Single codebase for Android & iOS; Expo speeds up build/OTA updates for a small team.
Doctor/Admin web portal	React + Vite, TypeScript, Tailwind CSS	Fast dev server, small bundle, shares component logic patterns with the mobile app.
Backend API	Node.js + Express, TypeScript	One language across client and server reduces context-switching for two developers.
Realtime layer	Socket.IO + Redis adapter	Simplest way to push live queue-number updates to many connected clients reliably.
Primary database	PostgreSQL + Prisma ORM	Strong relational integrity for queue/payment data; Prisma migrations keep schema changes fast and safe.
Cache / pub-sub	Redis	Backs Socket.IO scaling and caches "current queue number" for fast reads.
File/object storage	AWS S3 or Firebase Storage	Stores doctor certificates, patient reports, and prescriptions outside the database.
AI recommendation service	Python + FastAPI + scikit-learn (starting rule-based, upgrade to ML later)	Isolates experimental AI logic from the core booking system; easy to iterate independently.

Authentication	JWT (access + refresh tokens), bcrypt/argon2 password hashing, OTP via SMS gateway	Stateless auth scales horizontally; OTP verifies real phone numbers for both doctors and patients.
Payments	PayHere (Sri Lanka) and/or Stripe	Licensed local gateway (LKR) plus international fallback; avoids custom card handling.
Notifications	Firebase Cloud Messaging (push) + Twilio/local SMS gateway	Reliable delivery even when the app is closed.
Maps/Geolocation	Google Maps Platform (Places + Geocoding)	Doctor search by city/town and map display of dispensary location.
Hosting	Docker containers on Render/Railway (MVP) then AWS/GCP as usage grows	Low ops overhead initially; a clean migration path once the platform scales.
CI/CD	GitHub Actions	Automated lint/test/build/deploy pipeline maintainable by two developers.

7.3 Database Design & Connection

PostgreSQL is accessed exclusively through Prisma ORM from the Node.js backend, using a pooled connection (PgBouncer or Prisma's own connection pool) to handle bursts of queue-booking traffic without exhausting database connections. Configuration (host, port, credentials, pool size) is supplied via environment variables and never committed to source control; separate connection strings are used for local, staging, and production, matching the environments defined in the accompanying File & Development Structure annex.

- Base table: users (shared columns for all actor types) with a discriminator user_type column, extended by one-to-one doctors, patients, staff, admins tables, mirroring the EER generalisation in Figure 4.6.
- dispensaries table references doctors and stores city/town plus latitude/longitude for map search and "nearby doctor" queries (PostGIS extension optional if radius search is needed later).
- queue_tokens table is the hottest table at runtime; it is indexed on (dispensary_id, queue_date, token_number) to make "current number" lookups and "next available number" calculations fast.
- payments and invoices are linked 1:1 to a queue_token so every paid booking has a traceable financial record.
- medical_records and prescriptions carry a visibility/ownership scope enforced at the application layer (Premium feature) so only the owning patient and treating doctor can read them.
- reviews are linked to both doctor and patient, with a moderation_status column the admin can flip to hide abusive content without deleting the underlying record (audit-friendly).

Full table structures and cardinalities are shown in the ER diagram (Figure 4.5) and the specialisation hierarchy in the EER diagram (Figure 4.6).

7.4 Representative API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register a patient or doctor account; doctor registration also accepts certificate/licence file uploads.
GET	/api/doctors/search	Search verified doctors by name, city, town, or specialization.
POST	/api/queue/book	Book the next available queue token for a dispensary and date (idempotent).
PATCH	/api/queue/:dispensaryId/current	Doctor/staff updates the currently-serving queue number; broadcasts via Socket.IO.
POST	/api/payments/charge	Initiate an online payment for a booking through the payment gateway.

POST	/api/prescriptions	Doctor issues an e-prescription against a completed consultation.
POST	/api/reviews	Patient submits a rating/review after a completed visit.
POST	/api/admin/doctors/:id/ approve	Admin approves or rejects a doctor's submitted verification documents.
POST	/api/ai/recommend	Returns ranked doctor suggestions for patient-entered symptoms (calls the FastAPI service).

8. Delivery Plan for a Two-Developer Team

OneHealth is scoped as a rapid MVP, not a long-running project. The two developers are expected to work from a single monorepo with a clear role split, described fully in the companion document “OneHealth – File & Development Structure” (supplied as both a compiled PDF and an editable LaTeX .tex source). This section summarises the key decisions relevant to requirement delivery.

8.1 Role Split

Developer A — Backend & Data	Developer B — Frontend (Mobile + Web)
Node.js/Express API, PostgreSQL schema & Prisma migrations, authentication, queue logic, payment gateway integration, Socket.IO realtime layer, AI micro-service wiring, CI/CD and deployment.	React Native (Expo) patient/doctor app, React (Vite) web portal for doctor/admin, shared UI kit, API integration, push notification handling, and the booking/live-queue UX.

Both developers jointly own the shared TypeScript types/validation package, since it is the contract between frontend and backend.

8.2 Suggested Rapid Delivery Timeline

Phase	Focus
Week 1–2	Auth, base user/doctor/patient models, doctor registration with certificate upload, dispensary CRUD.
Week 3–4	Free-plan flow end-to-end: search, profile view, book token, live queue number via Socket.IO, review & rating.
Week 5–6	Standard plan: staff accounts & permissions, online payment, invoicing, e-prescription hand-off to pharmacist.
Week 7–8	Premium plan: patient medical records, doctor report upload/view, secure report sharing.
Week 9	AI recommendation service (symptom-to-doctor suggestion) using accumulated review/feedback data.
Week 10	Admin console (verification, moderation, billing), hardening, staging QA, production deployment.

This timeline targets a launch-ready Free + Standard experience within roughly six weeks, with Premium and AI features following as fast-follow releases rather than blocking the initial go-live — appropriate for validating the product with a small number of pilot dispensaries before wider rollout.

8.3 Reference

Full repository layout (monorepo folder tree for the mobile app, web portal, backend, AI micro-service, and infra), branching model, environments, CI/CD pipeline and coding conventions are documented in the companion annex: `OneHealth_FileStructure.tex` / `OneHealth_FileStructure.pdf`.

9. Appendix

9.1 Glossary (supplementary to Section 1.4)

- Free / Standard / Premium — the three doctor subscription packages defined in Section 3.
- Real-time queue number — the currently-being-served token number, pushed live to waiting patients.
- AI Recommender — a lightweight service that suggests doctors from patient-entered symptoms and historical review data; explicitly not a diagnostic tool.

9.2 Out of Scope for MVP

- Integration with national e-health/insurance systems.
- Video/tele-consultation (may be considered as a future package add-on).
- In-app pharmacy inventory management beyond marking a prescription as dispensed.

9.3 Approval

Role	Name	Signature / Date
Product Owner		
Developer A (Backend)		
Developer B (Frontend)		